THE HANDS-ON BEGINNER SERIES



FROM YOUR FIRST LINE OF CODE TO A LIVE DISCORD BOT

RUST DISCORD BOT ROADMAP.

A practical, mobile-first learning journey with Serenity, GitHub Codespaces, and Railway.

MADE FOR UNC | Revised edition · October 2026

MOBILE-FIRST / BEGINNER-FRIENDLY / ZERO ASSUMED EXPERIENCE

WELCOME / START HERE

Build something real. Learn the language along the way.

Hey UNC. You already did the hard part: deciding to learn something new. This guide is designed for a first-time programmer, not someone who already knows five languages. You can do it from a phone, and you do not need to understand every Rust feature before making progress.

01 LEARN — Rust basics ▶ **02** BUILD — Serenity bot ▶ **03** SHIP — Git + Railway ▶ **04** LEVEL UP — Better commands

Your finish line

You will write a small Discord bot that responds to **!ping**, **!hello**, and **!help**, store your code on GitHub, and deploy it as a background worker on Railway. The goal is not to memorize syntax. It is to **understand, test, fix, and ship**.

RECOMMENDED PATH

Browser + Codespaces

Best on iOS, Android, or a tablet
No local compiler to configure
Works with Railway + GitHub

ALTERNATIVE PATH

Android + Termux

Rust compiles on your phone
More setup and slower builds
Great if you like terminals

A small but important money note

Railway has a $0/month Free plan, but its $1 monthly usage credit is limited. The walkthrough gets your bot deployed on the free tier; it does **not** promise an always-online bot for $0 forever. We will show how to monitor usage and avoid a surprise bill.

THE MAP

Choose your next checkpoint.

Read in order the first time. Then return to whichever page solves the problem you are facing.

Set up your coding workspace 4

Learn programming in small pieces 6

Rust's superpower: ownership 8

Cargo, async, and your first project 10

Create your Discord application 12

Build the first working Serenity bot 14

Make changes without breaking things 16

Save the project to GitHub 17

Railway: the free-plan reality check 18

Prepare a deployment-ready Dockerfile 19

Deploy from GitHub to Railway 20

Operate, debug, and improve 21

A realistic four-week learning plan 23

Quick reference & trustworthy links 24

01 / START ON ANY DEVICE

Set up your coding workspace

Choose **Codespaces** if you only have a phone or tablet. It is a Linux development machine with an editor that runs in your browser. Use **Termux** if you have Android and want to compile directly on the device.

Option A — GitHub Codespaces (recommended)

- Open **github.com** and create a personal GitHub account, or sign in.
- Make a new repository named **unc-discord-bot**. Select **Private** while learning and check **Add a README**.
- Open the repository. Choose **Code → Codespaces → Create codespace on main**.
- When the browser editor appears, open **Terminal → New Terminal**. On small screens, landscape mode or a Bluetooth keyboard helps.
- Run the commands below. If Rust is already installed, you can skip the installer.

TERMINAL · CHECK THEN INSTALL RUST IF NEEDED

```
rustc --version
cargo --version

# Only if the commands above are missing:
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
. "$HOME/.cargo/env"
rustc --version
cargo --version
```

> **Understand the tools**
>
> **rustc** compiles Rust code. **Cargo** creates projects, downloads dependencies, runs checks, and builds programs. **Git** tracks versions. **GitHub** stores those versions online.

Make a runnable practice project now

Before doing the Rust exercises, create **src/main.rs** in the repository you opened in Codespaces. Run this **once** from its root directory:

CODESPACES · INSIDE YOUR REPOSITORY ROOT

```
cargo init --bin --name unc-discord-bot .
cargo run
```

If using **Termux** instead, after installing Rust run **cargo new unc-discord-bot**, then **cd unc-discord-bot** and **cargo run**. Both paths create the same starter files. You are now ready for Chapter 02.

Free Codespaces is a learning tool, not bot hosting

GitHub Free personal accounts include **120 core-hours/month** and 15 GB-month of storage (currently equivalent to about 60 hours on a two-core machine). Your codespace usually stops after 30 minutes of inactivity. It does **not** keep the bot online permanently. Source: GitHub billing and Codespaces docs.

- [ ] I can open the editor on my device.
- [ ] rustc --version displays a version.
- [ ] cargo --version displays a version.

Option B — Android + Termux

Get Termux from F-Droid or its official GitHub release. Avoid mixing Termux packages from different sources. Then run:

ANDROID / TERMUX

```
pkg update && pkg upgrade
pkg install rust clang git pkg-config openssl
rustc --version
cargo --version
```

Termux builds may be slower, and some crates may need extra native dependencies. If a mobile build becomes a wrestling match, switch to Codespaces: **learning Rust is the goal, not fighting the phone**.

Desktop, if available

Linux: install Git, curl, build-essential, and pkg-config through your package manager; install Rust with rustup. **Windows:** WSL2 is a convenient Linux-like setup. **macOS:** install Apple command-line tools before rustup. These are optional paths, not prerequisites for the mobile path.

02 / ZERO TO RUST

Learn programming in small pieces

If this is your first language, start with what programming is: **instructions + data + decisions + repetition**. Read code aloud. Change one line and predict what will happen before you run it.

Day 1 — Print, variables, and types

SRC/MAIN.RS · A FIRST RUST PROGRAM

```
fn main() {
    let name = "UNC";
    let mut level = 1;
    level += 1;
    println!("Hello, {name}! Level: {level}");
}
```

A **variable** holds information. **let** declares it; **mut** makes it changeable. **println!** prints text. **fn main()** is where this program starts. Rust infers many types, so explicit annotations are not always required. Copy short **let** snippets inside **main**, not at the top level of the file.

TRY DIFFERENT BASIC TYPES

```
let attempts: u32 = 3;       // unsigned integer
let temperature: f64 = 23.5; // decimal number
let online: bool = true;     // true or false
let greeting: &str = "Hi!";  // borrowed string slice
let owned = String::from("Hi!"); // owned text
```

> **Mini challenge 01**
>
> Print your name, your favorite number, and whether you like debugging. Change **level** three times, then explain why it needs **mut**.

Day 2 — Functions and decisions

A FUNCTION CAN ACCEPT DATA AND RETURN A VALUE

```
fn double(value: i32) -> i32 {
    value * 2
}

fn main() {
    let score = double(8);
    if score >= 10 {
        println!("Ready: {score}");
    } else {
        println!("Try again");
    }
}
```

A **function** is a reusable action. The final expression without a semicolon becomes its return value. An **if** expression selects which branch runs. Rust insists that a condition is a Boolean, not just a number.

Day 3 — Loops and collections

REPEAT WORK, STORE SEVERAL VALUES

```
fn main() {
    let commands = vec!["ping", "hello", "help"];

    for command in &commands {
        println!("Command: {command}");
    }

    println!("Total: {}", commands.len());
}
```

Vec is a growable list. **for** iterates over items. **&commands** borrows the collection instead of consuming it. That ampersand is a preview of Rust's most important idea.

> **Mini challenge 02**
>
> Create a list with five Discord command names. Print each one. Add a condition that reacts differently when the name equals **ping**.

03 / THE MENTAL MODEL

Rust’s superpower: ownership

Ownership lets Rust manage memory without a garbage collector. You do not need to master every lifetime rule today, but you **do** need to understand what moves and what borrows.

Own it, move it, borrow it

A MOVE: STRING VALUES OWN THEIR TEXT

```
let first = String::from("hello");
let second = first; // ownership moves to second
println!("{second}");
// println!("{first}"); // would NOT compile
```

A BORROW: READ WITHOUT TAKING OWNERSHIP

```
fn greet(name: &str) {
    println!("Welcome, {name}!");
}

fn main() {
    let username = String::from("UNC");
    greet(&username);
    println!("Still mine: {username}");
}
```

&T means an immutable reference, and **&mut T** means a mutable reference. You can have many readers, or one writer, but not both at once. This rule prevents a huge category of bugs.

Structs, enums, and matching

MODEL THINGS YOUR DISCORD BOT MIGHT UNDERSTAND

```
struct Player {
    name: String,
    level: u32,
}

enum BotCommand {
    Ping,
    Hello,
    Unknown,
}

fn reply(command: BotCommand) -> &'static str {
    match command {
        BotCommand::Ping => "Pong!",
        BotCommand::Hello => "Hello!",
        BotCommand::Unknown => "Try !help",
    }
}
```

A **struct** groups related data. An **enum** represents one of a fixed set of possibilities. **match** must handle every possible variant. Rust makes you be explicit, which is annoying in the best possible way.

Results and errors

NOT ALL OPERATIONS SUCCEED

```
fn parse_level(text: &str) -> Result<u32, String> {
    text.parse::<u32>().map_err(|e| e.to_string())
}

fn main() {
    match parse_level("10") {
        Ok(n) => println!("Level {n}"),
        Err(e) => println!("Invalid number: {e}"),
    }
}
```

Result<T, E> stores either **Ok(value)** or **Err(error)**. **Option<T>** stores **Some(value)** or **None**. Later, **?** lets a function return an error to its caller. Never hide an error by silently pretending everything worked.

> **Mini challenge 03**
>
> Add an enum with three bot states: **Starting**, **Online**, and **Offline**. Write a **match** that prints a different message for each state.

04 / MAKE IT REAL

Cargo, async, and your first project

Return to the project you created in Chapter 01

You already used **cargo init** (Codespaces) or **cargo new** (Termux). Do **not** run those commands a second time. Open that same project, which already contains **src/main.rs**.

TERMINAL · FROM THE PROJECT ROOT

```
pwd
cargo run

# A few commands you will use constantly:
cargo check
cargo fmt
cargo test
```

The project first printed “Hello, world!”. By now, **src/main.rs** should contain your own practice exercises from the preceding chapters. Save and run them with **cargo run**.

File or directory	Meaning
Cargo.toml	Project settings and dependencies
Cargo.lock	Exact dependency versions; commit it for an application
src/main.rs	The starting point of the program
target/	Compiled output; do not commit it

Why a Discord bot needs async

Discord communication depends on the network, and network operations may take time. **async** functions can pause at **.await** so the runtime can work on other tasks. **Tokio** is the async runtime. **Serenity** connects Rust code to the Discord API.

CONCEPTUAL ASYNC EXAMPLE

```
async fn fetch_answer() -> String {
    String::from("data arrived")
}

#[tokio::main]
async fn main() {
    let answer = fetch_answer().await;
    println!("{answer}");
}
```

> **Checkpoint before the bot**
>
> Can you explain **let**, **mut**, **fn**, **Vec**, **&**, **Result**, **match**, and **.await** in one sentence each? If not, practice those concepts first. You are not behind; you are building foundations.

Add the bot dependencies

Replace the dependency section in **Cargo.toml**. Keep the package name generated by Cargo. This guide targets Serenity **0.12.x**; the API can change in future releases.

CARGO.TOML · DEPENDENCIES

```
[package]
name = "unc-discord-bot"
version = "0.1.0"
edition = "2021"

[dependencies]
serenity = "0.12"
tokio = { version = "1", features = ["macros", "rt-multi-thread"] }
dotenvy = "0.15"
```

Run **cargo check**. The first download and compile can be slow. Future builds are usually faster because Cargo reuses previous work.

05 / GIVE IT AN IDENTITY

Create your Discord application

The Discord Developer Portal makes the bot account and generates the credential your program needs to sign in. Keep that credential private: anyone with the token could control the bot.

Create an application and bot

- Visit Discord Developer Portal and select **New Application**. Give it a simple name.
- Open **Bot** in the sidebar. New Discord applications normally already have a bot user; create one if your interface requires it.
- In **Bot → Privileged Gateway Intents**, enable **Message Content Intent** for this beginner example. Do not enable Server Members Intent: this bot does not use it.
- In **Bot**, use **Reset Token** or the available token-reveal action, then copy the new token somewhere private. Do not paste it in chat, screenshots, or GitHub.
- Under **Installation** or **OAuth2**, configure installation for a test server. Include the **bot** scope, and allow **View Channels**, **Send Messages**, and **Read Message History**.
- Use the install link to add the bot to a Discord server you manage. It may show as offline until you run the code.

> **Why turn on Message Content Intent?**
>
> Our starter uses **!ping**, which reads message text. Discord treats that as a privileged gateway intent. Later, you should learn **slash commands** (for example, /ping), which avoid this dependency for typical command handling.

- [ ] My Discord application has a bot user.
- [ ] Message Content Intent is enabled.
- [ ] The bot can be invited to a test server.

Add a local secret without leaking it

.ENV · NEVER COMMIT THE REAL VALUE

```
DISCORD_TOKEN=PASTE_YOUR_REAL_TOKEN_HERE
```

.GITIGNORE · IGNORE LOCAL SECRETS AND BUILD OUTPUT

```
/target
.env
.env.*
!.env.example
```

.ENV.EXAMPLE · SAFE EXAMPLE TO COMMIT

```
DISCORD_TOKEN=replace_with_your_own_token
```

Create all three files at the root of the project. **.env** is for local testing; the **Railway Variables** panel will provide the same environment variable in production.

> **Secret checkpoint**
>
> The real token is in **.env**, not in Rust source code. The real **.env** file must never appear in **git ls-files**.

06 / THE MOMENT OF TRUTH

Build the first working Serenity bot

Replace **src/main.rs** with the entire program below. It handles the “ready” event when Discord connects and responds to three message commands.

SRC/MAIN.RS · COMPLETE WORKING STARTER

```
use std::env;
use dotenvy::dotenv;
use serenity::async_trait;
use serenity::model::channel::Message;
use serenity::model::gateway::Ready;
use serenity::prelude::*;

struct Handler;

#[async_trait]
impl EventHandler for Handler {
    async fn ready(&self, _ctx: Context, ready: Ready) {
        println!("Logged in as {}", ready.user.name);
    }

    async fn message(&self, ctx: Context, msg: Message) {
        if msg.author.bot { return; }

        let reply = match msg.content.trim() {
            "!ping" => Some("Pong!".to_string()),
            "!help" => Some("Commands: !ping, !hello, !help".to_string()),
            "!hello" => Some(format!("Hello, {}!", msg.author.name)),
            _ => None,
        };

        if let Some(text) = reply {
            if let Err(err) = msg.channel_id.say(&ctx.http, text).await {
                eprintln!("Failed to send reply: {err}");
            }
        }
    }
}

#[tokio::main]
async fn main() {
    dotenv().ok();
    let token = env::var("DISCORD_TOKEN")
        .expect("Set DISCORD_TOKEN in .env or the environment");
    let intents = GatewayIntents::GUILD_MESSAGES
        | GatewayIntents::MESSAGE_CONTENT;

    let mut client = Client::builder(token, intents)
        .event_handler(Handler)
        .await
        .expect("Could not build Discord client");

    if let Err(err) = client.start().await {
        eprintln!("Discord client ended: {err}");
    }
}
```

At first, do **not** add databases, AI integrations, music playback, or 37 dependencies. One small bot working from end to end is a real achievement.

Run the loop: format → check → run

TERMINAL

```
cargo fmt
cargo check
cargo run
```

Look for **Logged in as YOUR_BOT_NAME** in the terminal. Then, in a server text channel where your bot has access, send **!ping**, **!hello**, and **!help**. The bot should reply. Keep the terminal open during local testing.

> **What does the bot actually do?**
>
> **EventHandler** receives Discord events. **msg.content.trim()** reads the message, **match** chooses a response, **Option** means "maybe reply," and **.await** sends the response without blocking the runtime.

07 / SMALL, SMART UPGRADES

Make changes without breaking things

Exercise A — Add a new command

Inside the existing **match** expression, add a branch **"!rust" => Some("Rust is awesome.".to_string()),**. Then add **!rust** to the help text. Test both commands.

Exercise B — Move command replies to a module

Create **src/commands.rs**. Start with a pure function that decides what text to send. The Discord handler can remain focused on receiving and sending messages.

SRC/COMMANDS.RS

```
pub fn response(content: &str, username: &str) -> Option<String> {
    match content.trim() {
        "!ping" => Some("Pong!".to_string()),
        "!hello" => Some(format!("Hello, {username}!")),
        "!help" => Some("Commands: !ping, !hello, !help, !rust".into()),
        "!rust" => Some("Rust is awesome.".into()),
        _ => None,
    }
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn ping_works() {
        assert_eq!(response("!ping", "UNC"), Some("Pong!".into()));
    }
}
```

At the very top of **src/main.rs**, add **mod commands;**. Replace the previous **let reply = match ...;** block with:

INSIDE HANDLER::MESSAGE

```
let reply = commands::response(&msg.content, &msg.author.name);
```

Leave the existing **if let Some(text) = reply** block untouched. Now run **cargo fmt**, **cargo test**, and **cargo run**.

Exercise C — Error messages are clues

When code fails, read the **first** meaningful compiler error rather than the last one. Search its exact error code in the Rust documentation. Change one thing, then run **cargo check** again. This is the debugging loop.

Checkpoint: !rust works, **cargo test** passes, and you can explain why moving reply logic into a pure function makes testing easier.

08 / VERSION CONTROL BEFORE DEPLOYMENT

Save the project to GitHub

A deployment platform needs your source code. GitHub is the bridge between your mobile editor and Railway. Your Git repository is also your undo button and your project history.

Make the first commit

If you created the repository in GitHub and opened it in Codespaces, the Git remote is already configured. Use the Source Control panel or the terminal commands below.

TERMINAL · CODESPACES OR A CONFIGURED GIT REPO

```
git status
git add Cargo.toml Cargo.lock src .gitignore .env.example
git commit -m "Build my first Discord bot"
git push
```

If Git asks for your identity, configure your name and email in its message. If **Cargo.lock** does not exist yet, run **cargo check** first. If **git push** fails outside Codespaces, connect your local Git repository to GitHub or use the Codespaces Source Control panel.

The safety check you should never skip

TERMINAL · CHECK WHICH FILES ARE TRACKED

```
git status --short
git ls-files
```

.env must NOT appear in the list. If you ever pushed a real bot token, **reset/rotate it immediately** in the Discord Developer Portal. Deleting a file from the latest commit does not erase it from old Git history.

> **Why Railway can deploy after this**
>
> Railway receives the GitHub repository (code + Cargo manifest + Dockerfile), builds an executable, and starts a container. Your phone is no longer required to stay online. GitHub stores code; Railway runs it.

09 / KNOW YOUR LIMITS

Railway: the free-plan reality check

You can deploy the bot using Railway's **Free** plan and the introductory **Trial**. Free hosting is usage-limited. For a continuously connected Discord bot, uptime depends on how much memory, CPU, and network it actually consumes.

Plan	Price	Included credit / key limits
Trial	US$ 0 initially	One-time US$ 5 credit; expires after up to 30 days.
Free	US$ 0/month	US$ 1 credit per month; 0.5 GB RAM and 1 vCPU per service.
Hobby	US$ 5/month	Paid subscription; US$ 5 monthly usage included.

> **Important: $0 deploy does not mean guaranteed 24/7 hosting**
>
> Railway meters usage. The **Free** credit can run out, and the bot can go offline when allowance or a usage limit is reached. Some Trial accounts have **restricted outbound network access** until GitHub verification; that can prevent a Discord gateway connection. Check your plan and verification status before debugging the Rust code.

Practical free-plan rules

- Use **one service / one replica** for your experiment. The bot needs a persistent background process, not a scheduled cron job.
- Keep **Serverless / App Sleeping off** for this gateway-connected bot; sleeping is not a reliable way to keep a Discord gateway connection.
- Do not add a database, volume, monitoring server, or public domain unless you actually need it.
- After deploying, open **Workspace → Usage** and inspect the real resource graph and estimated cost.
- Configure **usage alerts and a hard usage limit** if available on your plan. A hard limit deliberately takes services offline when reached.
- To remain at zero paid cost, choose the **Free** plan, do not upgrade to Hobby, and stop/remove your experiment when you are done.

As checked in October 2026. Pricing, eligibility, and product UI can change. Always verify Railway plans, free trial details, and cost controls before publishing your bot.

10 / PACKAGE YOUR RUST BOT

Prepare a deployment-ready Dockerfile

Railway can build projects automatically, but a small explicit **Dockerfile** gives you a dependable, teachable Rust deployment. The first image compiles the program; the second image contains only what is needed to run it.

Add a Dockerfile at the project root

DOCKERFILE · CAPITAL D, NO EXTENSION

```
FROM rust:1-bookworm AS builder
WORKDIR /app
COPY Cargo.toml Cargo.lock ./
COPY src ./src
RUN cargo build --release --locked

FROM debian:bookworm-slim
RUN apt-get update && apt-get install -y --no-install-recommends \
    ca-certificates && rm -rf /var/lib/apt/lists/*
COPY --from=builder /app/target/release/unc-discord-bot \
    /usr/local/bin/unc-discord-bot
CMD ["/usr/local/bin/unc-discord-bot"]
```

The output path comes from the Cargo package name **unc-discord-bot**. If you renamed the package, update both paths. **--locked** requires a committed **Cargo.lock**, which makes cloud builds more reproducible.

Protect your build context

.DOCKERIGNORE

```
target
.git
.env
.env.*
!.env.example
```

The last pattern keeps the harmless example file available if needed. The important rules are to exclude your real **.env** secret and the large **target** directory.

> **A Discord gateway bot is not a website**
>
> This bot connects **outbound** to Discord over HTTPS/WebSocket. It needs neither an HTTP server nor a public port. Do not invent a **PORT**, open a domain, or create a health-check endpoint just to satisfy a web-app tutorial.

11 / SHIP IT

Deploy from GitHub to Railway

You can do the entire deployment from a mobile browser. Keep your token nearby in a private password manager, not in a screenshot or message.

Follow this path

- Before opening Railway, push your deployment files to GitHub: run **git add Dockerfile .dockerignore Cargo.lock**, then **git commit -m "Add deployment"**, then **git push**.
- Go to railway.com and sign in. Connect the GitHub account that owns **unc-discord-bot**. Confirm which account plan and trial status you are using.
- Create a **New Project**, choose **Deploy from GitHub Repo** (wording may vary), and select **unc-discord-bot**. Authorize repository access if prompted.
- Railway should detect the root **Dockerfile**. Check the deployment logs for the Dockerfile build. If it picks a different builder, inspect the service build settings.
- Open your bot service → **Variables**. Add **DISCORD_TOKEN** with your real token as a secret environment variable. Save/apply it; allow the deployment to restart or redeploy.
- Under service settings, use **one replica**, leave **Serverless** disabled, and do not generate a public domain. The Dockerfile already defines the process start command.
- Open **Deployments → Logs**. Watch the image build, startup, and the message **Logged in as ...**. The app should appear online in Discord.
- From Discord, test **!ping** and **!hello**. Then open Railway's usage view and confirm you are inside the available credit.

> **How automatic updates work**
>
> If your Railway service stays connected to its GitHub branch, a future **git push** can trigger a new deployment. Observe the deployment logs each time; "successfully pushed" does not guarantee "successfully started."

If you only see a green deployment but the bot is offline

Read the **runtime logs**, not just the image build logs. Typical causes: the **DISCORD_TOKEN** variable was not set, the Discord token was rotated, Message Content Intent is disabled, the service hit a usage limit, or Railway's Limited Trial blocks required outbound connections.

12 / AFTER THE FIRST DEPLOY

Operate, debug, and improve

Troubleshooting without panic

Symptom	Likely cause	Try this
Bot offline	Service stopped, token missing, credit used	Check Railway runtime logs + usage
Bot online, no !ping	Intent or channel permissions	Enable Message Content; check server access
Build failed	Cargo.lock missing; wrong binary name	Run cargo check; compare Cargo.toml + Dockerfile
Starts, then exits	Token, network, or startup error	Read first error in runtime logs
Works locally, not on Railway	Local .env is not uploaded	Set DISCORD_TOKEN in Variables
Out-of-memory restart	Runtime limit exceeded	Remove extra crates/cache; inspect memory

Your healthy weekly routine

- Before coding: **git pull** if you use multiple devices.
- While coding: change one thing and run **cargo fmt**, **cargo check**, **cargo test**.
- Before deploying: verify **.env** is not tracked; make a Git commit.
- After deploying: test commands in Discord, then check Railway logs and usage.
- If your free credit is running low: stop/remove the Railway service until your next experiment.

What should you build next?

Level	Project	What it teaches
01	!roll 1–6	Randomness and parsing
02	!choose pizza | sushi	Collections and string handling
03	!uptime	Time and application state
04	Per-user cooldown	HashMap, duration, shared state
05	Slash commands	Modern Discord interactions
06	SQLite storage	Persistence, queries, data migrations

> **Modern Discord bots use slash commands**
>
> Prefix commands are an excellent first programming exercise, but a real bot should explore Discord interactions next. See Serenity's official examples, and consider **Poise** on top of Serenity for commands.

13 / PACE YOURSELF

A realistic four-week learning plan

This is an adjustable sequence, not a deadline. Repeat a week if you want more practice. The aim is real understanding rather than speedrunning compiler errors.

Week	Focus	Evidence you are ready
1	Workspace; variables; functions; if; for; Vec	You can write and change small Rust programs.
2	Ownership; borrowing; enums; Result; async; Cargo	You can explain errors and make a small module.
3	Discord app; Serenity; command routing; tests	Your bot responds locally to three commands.
4	Git; secrets; Dockerfile; Railway; operations	Your bot deploys and you monitor usage safely.

The 20-minute practice loop

- **5 min:** read one small example.
- **10 min:** type it, run it, change one behavior.
- **5 min:** write down what worked and one question you still have.

On days when time is short, just do the 10-minute build session. Tiny repetitions beat a giant weekend of copying code you do not understand.

Self-check quiz

- Why can a **String** be moved, but a borrowed **&str** does not take ownership?
- What is the difference between **Option** and **Result**?
- What does **.await** do in a Discord message handler?
- Why does the bot token belong in a Railway variable rather than **main.rs**?
- Why might a successfully deployed Railway service still be offline?
- What happens when the Free plan's usage credit is exhausted?

> **If you can answer these without looking**
>
> You are no longer just following a tutorial. You understand the essential pieces of your own software and can begin designing your next feature.

14 / KEEP LEARNING

Quick reference & trustworthy links

The commands you will actually reuse

CHEAT SHEET

```
cargo run             # compile and run
cargo check           # fast compile validation
cargo fmt             # format source code
cargo test            # run tests
cargo build --release # optimized executable

git status            # inspect changed files
git add .             # stage safe changes only
git commit -m "..."  # save a version
git push              # publish commits
```

Vocabulary in one line

Term	Meaning
Crate	A Rust library or application package
Cargo	Project and dependency manager
Borrow	Access a value without owning it
Result	Success or error in one explicit type
Async runtime	Schedules concurrent async tasks
Gateway intent	Which Discord events your bot requests
Docker image	Packaged environment containing your executable
Deployment	A running release of your application

Official links to bookmark

The Rust Programming Language — Start with the early chapters. Ownership is essential.

Rust by Example — Short, executable examples for each language feature.

Rustlings — Small exercises in an interactive loop.

Rust Playground — A lightweight experiment space on mobile.

Serenity documentation — API details for the version taught in this guide.

Serenity examples — See more complete Discord bot patterns.

Discord Developer Portal — Create and configure the bot account.

Railway: pricing + free trial — Verify current Free plan limits before deploying.

Railway: Dockerfiles — How Railway detects and builds Dockerfiles.

Railway: cost controls — Usage limits, alerts, and shutdown behavior.

GitHub Codespaces billing — Included core-hours, storage, and budget details.

> **A note about versions**
>
> This guide uses **Serenity 0.12** and reflects publicly documented Railway/GitHub limits checked in **October 2026**. APIs and plans change. Use the linked official documentation as the final authority.

YOU MADE IT / WHAT COMES NEXT

From “What is Rust?” to “My bot is online.”

You now have a path from a blank file to a real service. That includes the invisible parts beginners often skip: error handling, secrets, version control, resource limits, and deployment logs.

- [ ] I wrote real Rust and understood why it worked.
- [ ] I built a Discord bot with Serenity.
- [ ] I kept the token out of GitHub.
- [ ] I pushed the project to a repository.
- [ ] I deployed to Railway without assuming unlimited free hosting.
- [ ] I have one original feature I want to build next.

> **One last mission, UNC**
>
> Make one feature that is completely yours, however tiny. Name it. Code it. Break it. Fix it. Ship it. The language will start to feel familiar the moment the project becomes yours.

Keep the curiosity. Keep the commits. And when Rust gives you a compiler error, remember: it is trying to save Future You from a much worse afternoon.

BEGINNER → BUILDER / CONGRATULATIONS